

Shipment Creation Flow

Vendor ready-for-pickup, from packed parcel to courier airway bill

Zook backend · Jeebly integration · 10 September 2026

Before the call can succeed

Four facts must already be stored, written by four different steps. Each is refused rather than guessed at, and every refusal happens before anything is reserved or sent.

- **Payment terms, set at checkout.** The order carries a method of `prepaid` or `cod`, and a status only a server-side source may set to `paid`. Cash-on-delivery orders get a collection amount allocated to each parcel: its own sale price plus its share of the delivery fee, with the rounding remainder on the first parcel so the amounts sum exactly to the order total.
- **Packing started, both photos attached.** Uploading before and after stamps `photosVerifiedAt`.
- **A measured parcel weight**, recorded while the order is still `preparing`. That endpoint refuses once a shipment attempt is claimed, so the weight sent to the courier is always the stored one.
- **Structured address components.** The courier requires a pickup house number and landmark on the vendor, plus a house number and area on the delivery address. All four columns are nullable, so no existing row is invalidated.

Courier field requirements

Verified against the Jeebly demo API by probing each field with a deliberately invalid companion, so no unwanted shipments were created. Jeebly validates one field at a time, which is why refusals arrive one after another.

Requirement	Source in our schema
Weight must be a whole number; fractions and numeric strings are refused	<code>SubOrder.packageWeightKg</code> , rounded up at mapping time
Origin house number	<code>Vendor.pickupHouseNo</code>
Origin landmark	<code>Vendor.pickupLandmark</code>
Destination house number	<code>CustomerAddress.houseNo</code>
Destination area	<code>CustomerAddress.area</code>

On rounding. The vendor's measured weight stays in the database exactly as recorded. Only the chargeable figure sent to the courier is rounded up, which is how couriers bill regardless. A measured weight is never rounded down to zero.

Inside ready-for-pickup

1 Ownership and state

The vendor id comes from the token, never the request. A wrong owner is a not-found. An existing AWB, a status other than preparing, or a missing photo each return a conflict.

2 Recovery check

If a claim row already exists, the courier is never called again. With an AWB on the claim it jumps straight to step six. Without one it refuses as needing reconciliation, because the earlier outcome is unknown.

3 Facts and mapping, before anything is reserved

Weight, then payment state, then collection amount, read from stored columns only. A paid order ships prepaid even when placed as cash on delivery, so nothing is collected twice. The payload is built and fully validated here, so a data problem never leaves a claim behind.

4 Claim

A row keyed on the sub-order id is inserted as its own committed write. Its primary key is what stops two simultaneous requests, including across server instances, with no expiring lock.

5 One courier call

The sub-order is re-read under the committed claim and the payload rebuilt from that state, then sent once with no retries, a fifteen second deadline and redirects disabled.

6 Persist, then finish

The returned AWB is written onto the claim first, in its own write. Then one transaction sets the sub-order to ready with the courier name and AWB, and appends the history entry. Splitting those two writes is what makes a crash recoverable.

Failure behaviour

Only the first three steps leave no trace. From the claim onward it is kept deliberately, because a timeout or an unreadable response can follow a shipment that really was created.

Outcome	Response	Claim
Ownership, state, photos	404 or 409	none
Weight, payment, COD amount or address field missing	422 with a specific code	none
Credentials missing	503 <code>JEEBLY_NOT_CONFIGURED</code>	none
Courier rejects, times out, or returns a bad AWB	502 or 504	retained
AWB cannot be saved	503, identifiers logged	retained
Final transaction fails	503	retained with AWB

A retry after a rejection returns 409, asking for reconciliation. That is the system refusing to assume failure rather than a defect. Only an operator may clear a claim, and only once the courier has confirmed no shipment exists.

Where the AWB is stored

In two places, deliberately. `sub_orders.awb_number` is the business record that the vendor and admin screens read, and that guards against a second booking. `shipment_creations.awb_number` is the recovery record on the claim, written first so it survives a failure in the step that follows. A claim holding an AWB whose sub-order has none is precisely the recoverable state.

Verified end to end. These endpoints produced genuine airway bills against the Jeebly demo API: **JB114676**, **JB114677** and **JB114678**. Duplicate protection was confirmed by a second call on a booked sub-order, which returned 409 without contacting the courier.

Known gaps

- **No payment source writes `paid`**. Until a gateway confirmation or an authorised operator action sets it, only cash-on-delivery orders can ship. Defaulting the column or trusting the client's `paymentId` would defeat the check.
- **Existing rows need backfilling**. Every vendor and address created before the courier address columns existed has them empty, and cannot ship until they are filled. The 422 names the exact field to ask for. Placeholder values must never be invented: they would send a rider to an address that does not exist.
- **Delivery fee is always zero** at checkout, so COD allocation is currently just the sale price. The split logic is in place and unit-tested for when a real fee arrives.